Problem 09: AI-Based Code Repository Intelligence and Automated Software Maintenance Assistant

Domain: AI for Software Engineering | Code Intelligence | LLM Agents | Static Analysis
Difficulty: Hard

Problem

Large repositories have tangled dependencies, stale docs and duplicated logic. Developers spend much time understanding code, judging change impact, diagnosing bugs and picking tests.

Objective

Build an assistant that analyzes a real Git repository, traces dependencies, localizes bugs, proposes evidence-backed patches with tests, and verifies them in a sandbox.

Users

Developers, maintainers, code reviewers.

Core Requirements

- Accept a Git repo or source archive.
- Parse files; build a repository index.
- Extract functions, classes, imports, dependencies.
- Natural-language questions about the repo.
- Lexical + semantic code retrieval.
- Change-impact analysis for modified functions.
- Analyze bug reports and locate code.
- Generate patch with file references.
- Generate or select tests.
- Run tests/static checks in a sandbox and report results.

Advanced Challenges

- AST parsing + semantic retrieval.
- Dependency graph for impact analysis.
- No claims about code that was not retrieved.
- Patches consistent with existing interfaces.
- Efficient test selection.
- Regression detection via test runs.
- Patch correctness judged independently of the LLM.

Suggested Datasets

SWE-bench Lite, BugsInPy, Defects4J, CodeSearchNet, small open-source Python repos of your choice.

Suggested Tech

Python, tree-sitter, GitPython, ast/jedi, pyan or custom call graph, embeddings + FAISS/Qdrant, LangGraph, pytest, Docker sandbox, FastAPI.

Reference Architecture

Repo loader → tree-sitter parser → symbol table + call/import graph → hybrid index (BM25 + embeddings) → agent (localize → patch → test) → Docker sandbox runner → verifier (tests pass/fail, lint) → UI marking verified vs unverified.

Evaluation

Bug-localization accuracy (top-k file/function), retrieval precision, test-pass rate, regression rate, dependency-graph correctness vs ground truth, SWE-bench Lite subset resolve rate.

Security Note

Run untrusted repo code only in isolation (no network, resource limits). Never auto-apply patches to production.

Pitfalls
Trusting LLM self-assessment; sandbox escapes; leaking test files into the patch prompt.
Suggested Timeline (48h)
0-10h parsing + graph; 10-20h retrieval + Q&A; 20-32h localization + patching; 32-40h sandbox + tests; 40-48h eval + UI.

Standard Judging Rubric (applies to every problem)

Problem understanding	10%
Technical depth	20%
Data engineering (quality, leakage prevention, reproducibility)	10%
Model performance (metrics, baselines, honest evaluation)	15%
System architecture	15%
Working prototype	10%
Explainability and responsible AI	10%
Innovation and scalability	5%
Presentation and demo	5%

Mandatory Submission Checklist

- □ GitHub repository with setup instructions
- □ Working prototype or recorded demo
- □ Documented system architecture
- □ Dataset sources and preprocessing methodology
- □ Evaluation results with baseline comparisons
- □ Limitations and failure cases
- □ One technically challenging feature demonstrated
- □ Short presentation: problem, approach, results, future work